

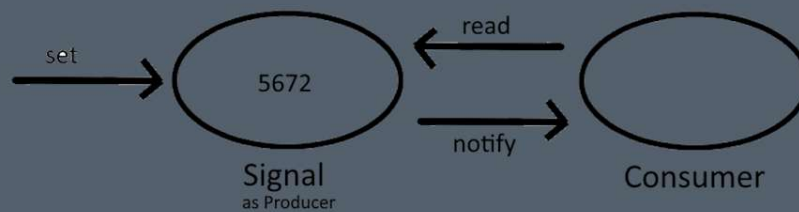
SIGNALS

© 2025 S&N Invent - Dominik Böcker, Dimitrij Ron

174

SIGNALS

- Angulars hauseigene Reaktivitätslösung sind die sog. Signals
- Vereinfacht gesagt ist ein Signal ein Wrapper um ein Wert oder Objekt
- Das Lesen der Werte eines Signals ist über die Getter-Funktion des Signals möglich
- Angular informiert alle Aufrufer bei Änderungen am Wert des Signals



© 2025 S&N Invent - Dominik Böcker, Dimitrij Ron

175

SIGNALS

- Signals können mithilfe der `signal()`-Funktion erstellt werden
- Dabei übergeben wir den Initialwert an die Funktion
- Den aktuellen Wert des Signals kann über die Getter-Funktion gelesen werden

```
const price = signal(0);  
console.log(price()); // Ausgabe: 0
```

© 2025 S&N Invent - Dominik Böcker, Dimitrij Ron

176

SIGNALS

- Mithilfe der `set()`-Funktion kann ein neuer Wert gesetzt werden
- Die `update()`-Funktion erleichtert es uns den aktuellen Wert zu manipulieren

```
const price = signal(0);
price.set(5);
console.log(price()); // Ausgabe: 5
price.update(alterPrice => price + 5);
console.log(price()); // Ausgabe: 10
```

SIGNALS READONLY

- Mit der `signal()`-Funktion erstellte Signals sind Writeable, d.h. ihr Wert kann an jeder Stelle beliebig geändert werden.
- Soll nur über geänderte Werte informiert werden, dann kann von einem Writeable-Signal ein Readonly-Signal abgeleitet werden.
- Dazu kann die `asReadonly()` Funktion verwendet werden

```
const price = signal(0);
const priceReadonly = price.asReadonly();
price.set(5); // Weiterhin erlaubt
priceReadonly.set(5); // Führt zum Fehler
```

SIGNALS COMPUTED

- Mithilfe der `computed()`-Funktion können abgeleitete Werte aus einem oder mehreren Signals erzeugt werden
- Das Computed-Signal gibt dabei den abgeleiteten Wert zurück

```
const price = signal(100);
const faktor = signal(5)

const priceMitFaktor = computed(() => price() + faktor())

console.log(priceMitFaktor()); // Ausgabe: 105
price.set(200);
console.log(priceMitFaktor()); // Ausgabe: 205
faktor.set(10);
console.log(priceMitFaktor()); // Ausgabe: 210
```

SIGNALS EFFECT

- Mithilfe der `effect()`-Funktion können Seiteneffekte ausgelöst werden
- Anders als `computed` geben Effekte keinen Wert zurück

```
const price = signal(100);

const priceEffect = effect(() => {
  console.log('Neuer Preis: ', price());
});

price.set(200); // Führt zu Ausgabe: Neuer Preis 200
price.set(150); // Führt zu Ausgabe: Neuer Preis 150
```

SIGNALS EFFECT

- Standardmäßig können Effekte nur innerhalb des Injection-Contexts erstellt werden (im Kontruktor oder bei der Attributdeklaration)

```
@Component({...})
export class App {
  readonly price = input.required<number>();

  readonly loggingEffect = effect(() => {console.log('Aktueller Preis:', this.price());});

  constructor() {
    effect(() => {console.log('Aktueller Preis:', this.price());})
  }
}
```

SIGNALS EFFECT

- Wenn ein Effekt außerhalb des Injection-Context erzeugt werden soll, muss der Injector mit übergeben werden

```
@Component({...})
export class App {
  readonly price = input.required<number>();
  readonly injector = inject(Injector);

  initialisiereLogging(): void {
    effect(() => {
      console.log('Aktueller Preis: ${this.price()}');
    }, {injector: this.injector});
  }
}
```

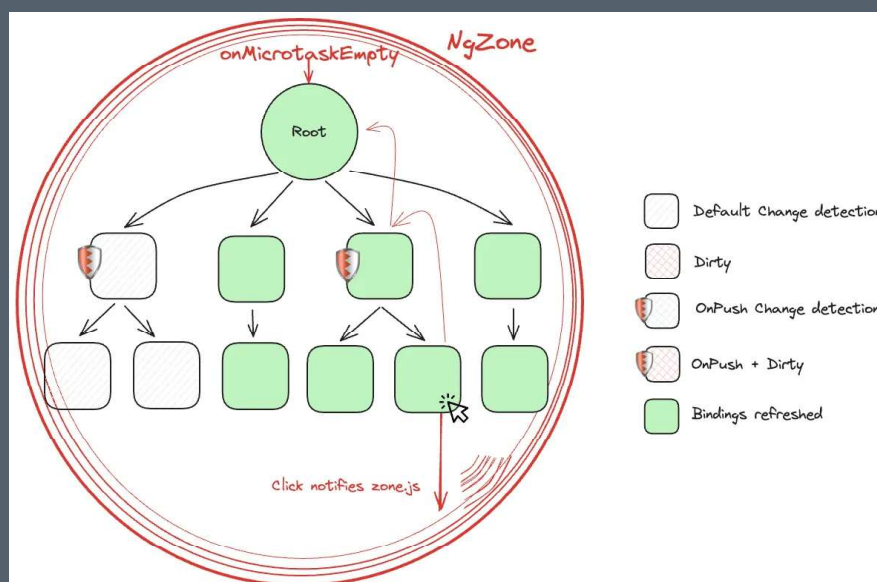
CHANGE DETECTION (BISHER) - ZONE.JS

- Bisher verwendete Angular Zone.js um über mögliche Änderungen informiert zu werden
- Zone.Js Monkey-patched dazu alle Event-Handler
- Jeder ausgeführte Event-Handler triggert Angulars Change-Detection

CHANGE DETECTION (BISHER) - ZONE.JS



CHANGE DETECTION (BISHER) - ZONE.JS REFRESH

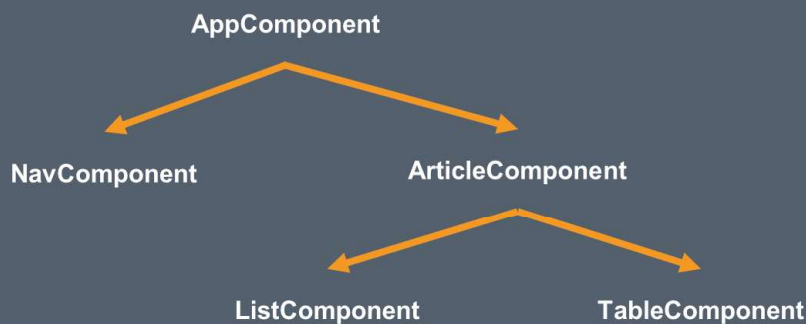


CHANGE DETECTION (BISHER) - ZONE.JS NACHTEILE

- Zone.js bringt overhead in unser Bundle (100kb)
- Zone.js fehler sind sehr schwer zu identifizieren
- async und await können nicht gepatched werden und müssen im Build zu Promises konvertiert werden
- Bei Änderungen muss die komplette Komponente sowie ihre Vorgänger im Komponentebaum gecheckt werden

CHANGE DETECTION - MIT SIGNALS

- Signals ermöglichen reaktive Updates ohne Zone.js
- Überwachen granular Änderungen am Datenmodell
- UI wird nur an den betroffenen Stellen aktualisiert
- Reduziert Performance-Overhead von globaler Change Detection



SIGNALS - INTEROPERABILITÄT MIT RXJS

- Signals können zu Observables und Observables zu Signals konvertiert werden
- Das ist hilfreich wenn wir mit den RxJs-Operatoren auf einem Signal arbeiten wollen

```
@Component({})
export class CountComponent {
  readonly a = signal(12);

  readonly a$ = toObservable(this.a);
  readonly b$ = this.a$.pipe(map((a) => a + 2));

  readonly c = toSignal(this.a$);
}
```

HTTPRESOURCE

- Seit Angular 19.2 steht die `httpResource` als Signal-basierte Alternative zur Verfügung

```
export class ProductService {  
  getList(): HttpResourceRef<Product[]> {  
    return httpResource<Product[]>(  
      ()=>({  
        url: this.api,  
        method: 'GET',  
      }  
    ),  
      {  
        defaultValue: []  
      }  
    );  
  }  
}
```

HTTPRESOURCE

- Die `httpResource` hält die Antwort des Servers als Signal vor
- Das Antwort-Signal ist im `value`-Attribut der `httpResource` zu finden
- Auf Änderungen des `value`-Signal kann nun reaktiv reagiert werden

```
export class ProductList {  
  readonly productsResource = inject(ProductService).getList();  
  readonly totalPrice = computed(() => this.productsResource.value().map((product) => product.price)  
    .reduce((prev, curr) => prev + curr, 0));  
}
```

HTTPRESOURCE

- Die `httpResource` enthält weitere Signals, z.B. im Attribut `loading`
- Solange der zugrundeliegende Request noch lädt, enthält es den Wert `true`

```
@if(productsResource.isLoading()) {  
  <mat-progress-bar mode="indeterminate"/>  
} @else {  
  <!-- Anzeige der Produkte -->  
}
```

HTTPRESOURCE

- Der httpResource können Werte als Signal übergeben werden
- Ein ändern der Werte sorgt für ein erneutes Senden des Requests

```
export class ProductList {
  readonly nameFilter = signal('');
  readonly productsResource = inject(ProductService).getList(this.nameFilter);
}
```

```
<input matInput [(ngModel)]="nameFilter" />
```

```
export class ProductService {
  getListFiltered(nameFilter: Signal<string>): HttpResourceRef<Product[]> {
    return httpResource<Product[]>({
      ()=>({
        url: this.api, method: 'GET',
        params: {
          name: nameFilter()
        }
      }),
      { defaultValue: [] }
    });
  }
}
```

HTTPRESOURCE

- Der Request kann aber auch händisch neu angestoßen werden über die Reload-Funktion
- Das ist vor allem interessant wenn bekannt ist das sich die Daten auf dem Server geändert haben.

```
export class ProductList {
  readonly nameFilter = signal('');
  readonly productsResource = inject(ProductService).getList(this.nameFilter);

  onSaveProduct() {
    this.productsResource.reload();
  }
}
```

FALLSTRICKE

- Versehentliches Entpacken
 - Man arbeitet mit dem aktuellen Value statt mit dem Signal

```
class TestComponent {
  readonly signalA = signal(1);

  constructor() {
    let a = this.signalA();
    console.log(a);
    a += 5;
    console.log(a);
    console.log(this.signalA());
  }
}
```

FALLSTRICKE

- Rekursion
 - Selbsterklärend

```
class TestComponent {
  readonly signalA = signal(1);

  constructor() {
    effect(() => {
      const a = this.signalA();
      this.signalA.set(a += 5);
    });
  }
}
```

FALLSTRICKE

- Bedingungen in effect / computed
 - Jeder Durchlauf bestimmt die Subscriber
 - Bedingungen können Signals "abklemmen"

```
@Component({
  selector: 'test',
  template: `<input (change)="signalA.set(input1.value)" type="number" #input1/>` +
    `<input (change)="signalB.set(input2.value)" type="number" #input2/>` +
    `<input (change)="signalC.set(input3.value)" type="number" #input3/>`
})
class TestComponent {
  readonly signalA = signal(1);
  readonly signalB = signal(2);
  readonly signalC = signal(3);

  constructor() {
    effect(() => {
      if (this.signalA() + this.signalB() > 5) {
        console.log(this.signalA() + this.signalB() + this.signalC());
      }
    });
  }
}
```